

Basic IPC

2026 Semester 2 COMPSCI 340: Operating Systems
Talía Xu

Lecture 6
2.2.3

Based on original slides by Professor Jon Eyolfson.

This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/)

Separate Processes Need a Controlled Way to Communicate

Each process has its own address space and resources.

IPC creates a controlled path across that boundary:

- a **pipe** moves bytes from one process to another
- a **signal** tells a process that an event occurred

Other IPC mechanisms exist, including shared memory and sockets. This lecture develops pipes and signals because they show the two basic ideas clearly: **moving data** and **delivering an event**.

This Lecture Uses One Sender and One Receiver

Mechanism	Situation in this lecture	Direction
Regular pipe	A parent and child share pipe descriptors	One writer → one reader
Named pipe	Two processes open the same filesystem name	One writer → one reader
Signal	The kernel notifies one user process	Kernel → process

Pipes can have multiple readers or writers, but ownership and coordination become less clear. We begin with the common producer–consumer pattern.

A Pipe Is a Kernel-Managed Byte Stream

```
int fd[2];    pipe(fd);
```

- `fd[0]` is the read end
- `fd[1]` is the write end
- The kernel holds written bytes until the reader receives them
- Bytes leave the pipe in the same order they entered
- A pipe is a stream; it does not preserve application message boundaries

In this lecture, one process writes and one process reads.

A Regular Pipe Usually Connects Related Processes

1. pipe returns two descriptors only to the calling process
2. The pipe has no filesystem name for another process to open
3. The process calls fork
4. The child inherits descriptors for the same pipe

Our case: the parent writes and its child reads.

This is not a hard kernel restriction. Any process that inherits or receives an endpoint descriptor can use the pipe. Parent-child is simply the direct, usual setup.

Create the Pipe Before fork

Process	Read descriptor	Write descriptor
Before fork	fd[0]	fd[1]
Parent after fork	Copy of fd[0]	Copy of fd[1]
Child after fork	Copy of fd[0]	Copy of fd[1]

Both descriptor tables refer to the same kernel pipe. Each process then closes the endpoint it does not use.

Trace One Message from Parent to Child

```
int fd[2];
pipe(fd);

pid_t pid = fork();

if (pid == 0) {
    char buf[32];
    close(fd[1]);
    ssize_t n = read(fd[0], buf, sizeof buf - 1);
    if (n > 0) {
        buf[n] = '\0';
        printf("child: %s\n", buf);
    }
    close(fd[0]);
    return 0;
}

close(fd[0]);
write(fd[1], "hello", 5);
close(fd[1]);
waitpid(pid, NULL, 0);
```

Each Process Keeps Only the Endpoint It Uses

1. pipe creates the read and write descriptors
2. fork gives both descriptors to the child
3. The child closes the write end, then reads
4. The parent closes the read end, then writes hello
5. The kernel copies the bytes into the child's buffer
6. Both processes close their remaining descriptors when finished

The child prints `child: hello`. Closing unused endpoints makes the intended ownership visible in the code.

Always Check How Many Bytes Moved

- `write(fd, buf, count)` returns the number of bytes written
- `read(fd, buf, count)` returns the number of bytes read
- `read` may wait while the pipe contains no data
- `write` may wait when the kernel's pipe buffer is full
- A larger transfer may require several calls

The small example sends only five bytes. General code checks return values and repeats the operation until the required data has moved.

A Named Pipe Lets Unrelated Processes Find the Channel

A regular pipe—also called an unnamed pipe—is reached through inherited descriptors. A **named pipe** is reached through a pathname.

	Regular pipe	Named pipe
Created with	pipe()	mkfifo()
Found through	inherited descriptor	filesystem pathname
Typical pair	related processes	processes started separately

The pathname identifies the channel. The bytes still travel through a kernel pipe buffer; they are not stored in a regular file.

Open the Same Named Pipe from Two C Programs

Create the channel once with `mkfifo("events", 0600);`.

Reader

```
int fd = open("events", O_RDONLY);

char buf[16];
ssize_t n = read(fd, buf, sizeof buf);

if (n > 0)
    write(STDOUT_FILENO, buf, n);

close(fd);
```

Writer

```
int fd = open("events", O_WRONLY);

write(fd, "ready\n", 6);

close(fd);
```

The Name Replaces the fork Connection

1. mkfifo creates the pathname events
2. The reader opens that pathname with O_RDONLY
3. The writer opens the same pathname with O_WRONLY
4. The writer sends ready; the reader receives those bytes
5. Each process closes its descriptor when finished

With ordinary blocking I/O, opening one endpoint may wait until the other endpoint is opened. The two programs do not need a parent-child relationship.

Pipes Carry Data; Signals Announce Events

Pipe or named pipe

- Carries a byte stream
- Reader explicitly calls read
- Useful for producer–consumer data

Signal

- Carries an event number
- May interrupt normal execution
- Useful for notification or control

A signal says that something happened; it is not a general data channel.

The Kernel Delivers a Signal to a User Process

When a signal is delivered, the process can:

- accept the signal's default action
- ignore many kinds of signal
- run a handler installed by the program

The event may begin with hardware, a terminal, a timer, or another process, but the kernel performs delivery to the target process.

SIGKILL and SIGSTOP cannot be caught or ignored.

A Small SIGINT Handler Can Stop the Process

```
static void on_sigint(int signo) {           // Signal handler
    (void) signo;                           // No signal details are needed
    const char message[] = "stopping\n";    // Shutdown message
    write(STDOUT_FILENO, message,          // Safe output from the handler
          sizeof message - 1);
    _exit(0);                               // Safe immediate shutdown
}

int main(void) {
    struct sigaction action = {0};          // Signal-handling configuration
    action.sa_handler = on_sigint;          // Use our signal handler
    sigemptyset(&action.sa_mask);          // Block no additional signals
    sigaction(SIGINT, &action, NULL);       // Register it for Ctrl+C

    for (;;)                                // Wait indefinitely
        pause();                            // Sleep until an event arrives
}
```

A Signal Handler Runs Outside the Normal Control Flow

A signal may arrive between ordinary instructions, so a handler must stay small and predictable.

In the example:

- `sigaction` tells the kernel which function handles `SIGINT`
- `pause` sleeps instead of repeatedly checking for an event
- `Ctrl+C` causes the kernel to deliver `SIGINT`
- the handler uses only `write` and `_exit`, which are safe in this context

Processes Can Wait by Blocking, Polling, or Receiving a Signal

Approach	What the process does	Trade-off
Blocking	Sleeps inside a call until work is ready	Simple and avoids wasting CPU
Polling	Repeats a non-blocking check	Can combine checks with other work, but frequent checks waste CPU
Signal	Runs a handler when the event is delivered	Prompt notification, but handlers are restricted

Kernel Events Commonly Become User-Process Signals

Situation	What the kernel observes	Signal delivered
User presses Ctrl+C	terminal input	SIGINT
Process accesses invalid memory	CPU exception	SIGSEGV
Child changes state	process-state change	SIGCHLD
Process timer expires	timer event	SIGALRM

The common direction is **event** $\longrightarrow$ **kernel** $\longrightarrow$ **user process**. A process may also ask the kernel to send a signal to another process, for example with `kill`.

Summary: IPC Either Moves Data or Delivers an Event

- A regular pipe is a one-way byte stream reached through descriptors
- A parent commonly creates the pipe before fork so its child inherits both endpoints
- This lecture uses the simple one-writer, one-reader pattern
- A named pipe adds a pathname, allowing unrelated processes to connect
- A signal is an event notification delivered by the kernel to a process
- Signal handlers must stay small and use safe operations